

Simulation de variables aléatoires

David Darras

20/03/2025

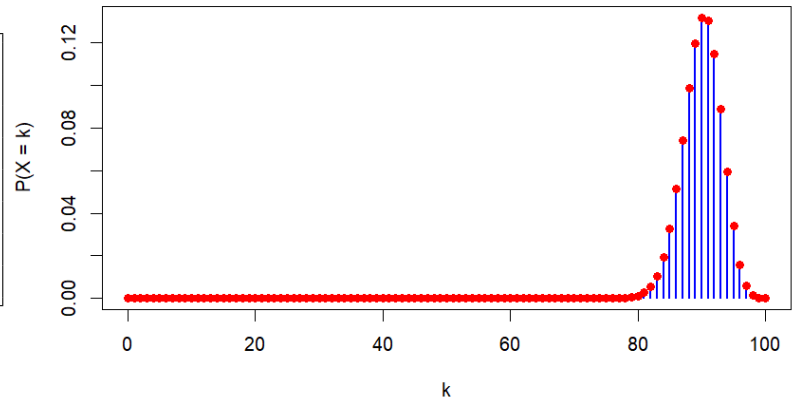
1 Introduction

Soit X une variable aléatoire discrète ou continue. L'objectif est de simuler cette variable aléatoire, c'est-à-dire d'écrire une fonction qui génère un nombre dans $X(\Omega)$ en respectant la distribution de probabilité associée à X . Pour ce faire, on dispose initialement d'un générateur aléatoire donnant un nombre flottant compris entre 0 et 1, c'est-à-dire simulant une variable aléatoire $U \sim \mathcal{U}([0, 1])$. L'un des plus connus est le *Mersenne Twister*, utilisé avec les simulations de Monte Carlo en raison de sa grande période.

Exemple intuitif : Si je crée une fonction `generate_X` telle que $X \sim \mathcal{B}(100, 0.9)$, alors les valeurs de retour de la fonction seront dans l'ensemble $X(\Omega) = \{0, \dots, 100\}$, et on aura une plus forte chance d'obtenir la valeur 90 plutôt que 10, par exemple, car $P(X = 90) \gg P(X = 10)$.

Loi Binomiale: B(100, 0.9)

```
1 n <- 100
2 p <- 0.9
3 x <- 0:n
4 y <- dbinom(x, size=n, prob=p)
5 plot(x, y, type="h", lwd=2, col="blue",
6      main="Loi Binomiale: B(100, 0.9)",
7      xlab="k", ylab="P(X = k)")
8 points(x, y, col="red", pch=19)
```



2 Simulation d'une variable aléatoire discrète

2.1 Algorithme

```
1: Arrange  $x_0, x_1, \dots, x_k$  s.t.  $x_0 < x_1 < \dots < x_k$ .
2: Generate  $u \in U(0, 1)$ 
3: for  $i = 0$  to  $k$  do
4:   if  $\sum_{j=0}^{i-1} p_j < u \leq \sum_{j=0}^i p_j$  then
5:     Return  $x_i$ 
6:   end if
7: end for
```

2.2 Exemple n°1

Simulons la variable aléatoire discrète X telle que $X(\Omega) = \{1, 5, 7, 9\}$ et $P(X = 1) = 0.3$, $P(X = 5) = 0.1$, $P(X = 7) = 0.5$, $P(X = 9) = 0.1$.

Rappel : Pour que la loi soit correcte, il faut que $\forall k \in X(\Omega), 0 \leq P(X = k) \leq 1$ et $\sum_{k \in X(\Omega)} P(X = k) = 1$.

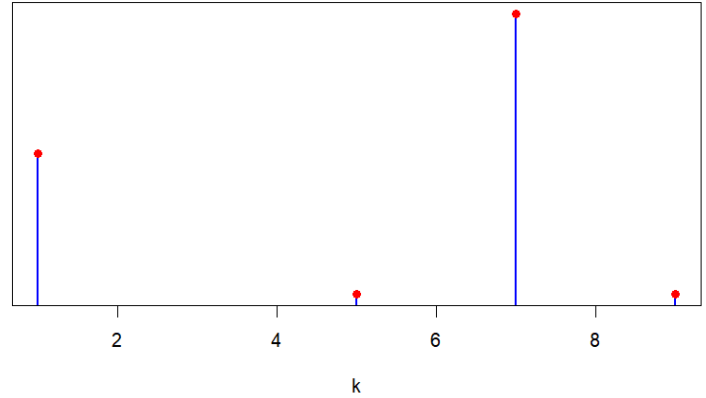
Distribution de probabilité

```

1 generate_X <- function( X_Omega, P ) {
2   u <- runif( 1 )
3   k <- 1
4   sup <- P[ k ]
5   repeat {
6     if( u <= sup )
7       return( X_Omega[ k ] )
8     k <- k + 1
9     sup <- sup + P[ k ]
10  }
11 }

```

P(X=k)



2.3 Exemple n°2

Soit $X \sim \mathcal{G}(p)$. Alors, $X(\Omega) = \mathbb{N}^*$ et pour tout $k \in X(\Omega)$, $P(X = k) = (1 - p)^{k-1}p$. Pour l'exemple, on pose $p = 0.01$.

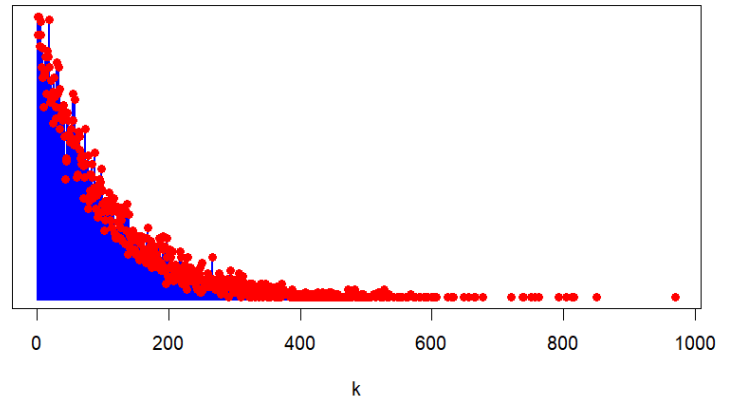
```

1 generate_X <- function() {
2   P <- function( k ) {
3     p <- 0.01
4     return( (1 - p)^(k-1) * p )
5   }
6   u <- runif( 1 )
7   k <- 0
8   sup <- P( k )
9   repeat {
10    if( u <= sup )
11      return( k )
12    k <- k + 1
13    sup <- sup + P( k )
14  }
15 }

```

Distribution de probabilité

P(X=k)



3 Simulation d'une variable aléatoire à densité

3.1 Méthode de la transformée inverse

La méthode s'applique uniquement si l'on peut calculer F_X^{-1} , la réciproque de la fonction de répartition F_X . Si $U \sim \mathcal{U}([0, 1])$, alors $F_X^{-1}(U)$ suit la même loi que X .

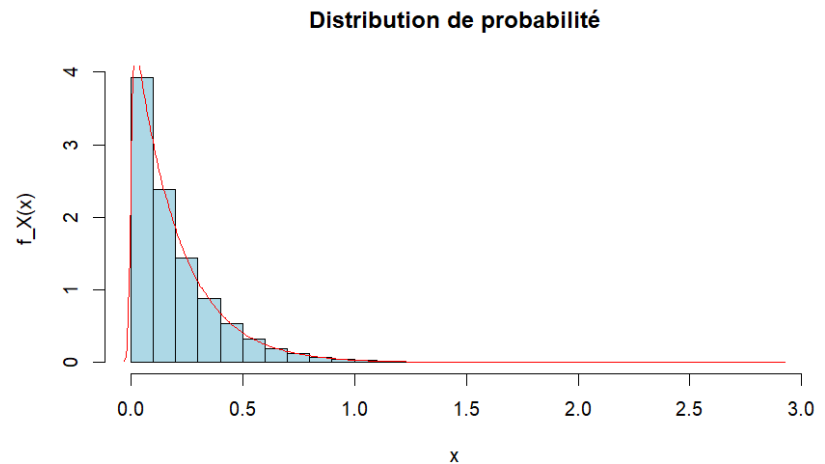
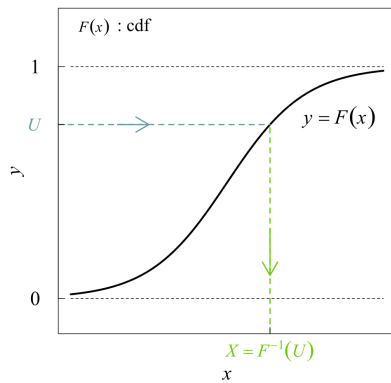
Exemple : Soit $X \sim \mathcal{E}(\lambda)$. Alors $X(\Omega) = \mathbb{R}^+$ et pour tout $x \in X(\Omega)$, $F_X(x) = 1 - e^{-\lambda x}$.

Donc, pour tout $u \in [0, 1]$, $F_X^{-1}(u) = -\frac{1}{\lambda} \ln(1 - u)$. Dès lors, $\frac{-\ln(U)}{\lambda} \sim \mathcal{E}(\lambda)$ puisque $1 - U \sim \mathcal{U}([0, 1])$.

```

1 generate_X <- function() {
2   lambda <- 3
3   u <- runif(1)
4   return( -log(u) / labmda )
5 }

```



3.2 Méthode de rejet

3.2.1 Algorithme

1. Trouver une variable aléatoire Y facile à simuler (comme la loi uniforme) telle que pour tout $t \in \mathbb{R}$, $f_X(t) \leq c \times f_Y(t)$ et $f_X(t) = 0 \iff f_Y(t) = 0$.
2. Générer $u \in \mathcal{U}([0, 1])$ et $t \in Y(\Omega)$.
3. Si $u > \frac{f_X(t)}{c \times f_Y(t)}$, on rejette la solution et on repart au point 2.
4. Sinon, on renvoie $t \in X(\Omega)$.

Remarque : Le nombre d'itérations N suit une loi géométrique de paramètre $\frac{1}{c}$. Dès lors, le nombre moyen d'itérations est $\mathbb{E}[N] = c$. On veut donc choisir c le plus petit possible, pour être le plus performant.

3.2.2 Exemple n°1

Soit une variable aléatoire à densité X telle que $X(\Omega) = [0, 1]$ et pour tout $x \in X(\Omega)$, $f_X(x) = 20x(1-x)^3$.

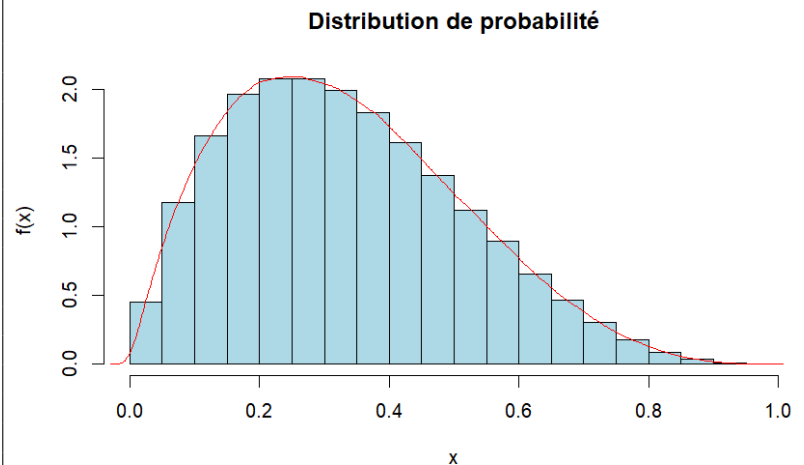
Rappel : Pour que la loi soit correcte, il faut que pour tout $x \in \mathbb{R}$, $f_X(x) \geq 0$ et $\int_{\mathbb{R}} f_X(x) dx = 1$.

On choisit une variable facile à simuler qui recouvre le même support que X , soit $Y \sim \mathcal{U}([0, 1])$. Dès lors, $Y(\Omega) = [0, 1]$ et pour tout $y \in Y(\Omega)$, $f_Y(y) = 1$. Pour trouver le c le plus petit pour que f_Y englobe f_X , on pose $c = \max_{t \in \mathbb{R}} \frac{f_X(t)}{f_Y(t)}$. Après une rapide étude des variations de la fonction, on trouve $c = \frac{135}{64} \approx 2.1$.

```

1  f_X <- function(x) {
2    if( 0 <= x && x <= 1 )
3      return( 20 * x * (1 - x) ^ 3 );
4    return(0);
5  }
6  f_Y <- function(y) {
7    if( 0 <= y && y <= 1 ) return(1);
8    return(0);
9  }
10 generate_Y <- function() {
11   return(runif(1));
12 }
13 generate_X <- function() {
14   c = 135/64;
15   repeat {
16     u = runif(1);
17     t = generate_Y();
18     if( u <= f_X(t) / ( c * f_Y(t) ) )
19       return(t);
20   }
21 }

```



3.2.3 Exemple n°2

Approximons la valeur de π avec la méthode de rejet. On se donne X une variable aléatoire de densité telle que $X(\Omega) = \{(x_1, x_2) \in [0, 1]^2 \mid x_1^2 + x_2^2 \leq 1\}$ et pour tout $\forall \vec{x} \in X(\Omega), f_X(\vec{x}) = 1$.

Remarque : $X(\Omega)$ représente l'ensemble de tous les points dans un quart de cercle d'aire $\frac{\pi}{4}$.

```
1 f_X <- function(x1, x2) {
2   if( x1 >= 0 && x2 >= 0 && x1^2 + x2^2 <= 1 )
3     return(1)
4   return(0)
5 }
6 f_Y <- function(y1, y2) {
7   if( 0 <= y1 && y1 <= 1 && 0 <= y2 && y2 <= 1 )
8     return(1)
9   return(0)
10 }
11 generate_Y <- function() {
12   return(c(runif(1), runif(1)))
13 }
14 generate_Pi <- function(N) {
15   c <- 1
16   accepted <- 0
17   rejected <- 0
18   for(i in 1:N) {
19     u <- runif(1);
20     t <- generate_Y();
21     if( u <= f_X(t[1], t[2]) / ( c * f_Y(t[1], t[2]) )
22       accepted <- accepted + 1
23     else
24       rejected <- rejected + 1
25   }
26   return( 4 * accepted / (accepted + rejected) )
27 }
28
29 print(generate_Pi(1000000)) # 3.14458
```

points acceptés (verts) et rejetés (rouges)

